

Rancangan SSO — SSB sebagai Identity Provider (IdP)

Status: **Tahap 1-3 SELESAI + Tahap 4 paket integrasi ESS disiapkan** (IdP terverifikasi siap; penerapan di repo ESS) — Tahap 5 (Warehouse + logout) berikutnya Tanggal: 2026-06-06 Aplikasi utama (IdP): **SSB-PROJECT** (Laravel 8 / PHP 8) Client awal (SP): **ESS, Warehouse**

1. Tujuan

Menjadikan **SSB-PROJECT** sebagai pusat **Single Sign-On (SSO)** sehingga aplikasi lain (ESS, Warehouse, dan app berikutnya) bisa memakai satu kali login. Implementasi harus **bersifat additive** — tidak mengganggu proses login, API, dan integrasi yang sudah berjalan.

Keputusan arsitektur (sudah disepakati)

Topik	Keputusan
Protokol	OAuth2 / OIDC via Laravel Passport
Manajemen role	SSB hanya autentikasi (siapa user). Role/otorisasi dikelola lokal di tiap app.

2. Kondisi aplikasi saat ini (baseline)

- **Framework:** Laravel 8, PHP 8.0
- **Auth web:** session-based (`laravel/ui` , `LoginController`) — login pakai **NIK + password**
- **Auth API:** **Laravel Sanctum** (`AuthController@login` → Bearer token)
- **Otorisasi:** Spatie Permission (roles & permissions)
- **Integrasi eksternal yang sudah ada:**
 - `ServiceTokenController` — token permanen untuk aplikasi eksternal
 - `MediaController` — serve file ke aplikasi eksternal (butuh service token)
- **Master karyawan via API eksternal** (nama di-snapshot, bukan tabel lokal)

Semua hal di atas **TETAP berjalan** dan tidak diubah oleh rancangan ini.

3. Prinsip: tanpa mengganggu proses existing

Yang TETAP jalan (tidak disentuh)	Yang DITAMBAH (baru, terpisah)
Login web session (<code>LoginController</code>)	Endpoint OAuth2: <code>/oauth/authorize</code> , <code>/oauth/token</code>
<code>AuthController@login</code> (Sanctum token)	Endpoint <code>/oauth/userinfo</code>
<code>ServiceToken</code> & <code>MediaController</code>	Halaman login SSO (reuse session existing)
Guard <code>web</code> & <code>api</code> (Sanctum)	Guard baru <code>oauth</code> (Passport), berdampingan
Tabel <code>personal_access_tokens</code> (Sanctum)	Tabel <code>oauth_*</code> (Passport) + <code>sso_clients</code>

Strategi: **route baru, tabel baru, controller baru, guard baru**. Flow lama nol perubahan, sehingga setiap tahap bisa di-rollback tanpa risiko.

4. Pembagian tanggung jawab

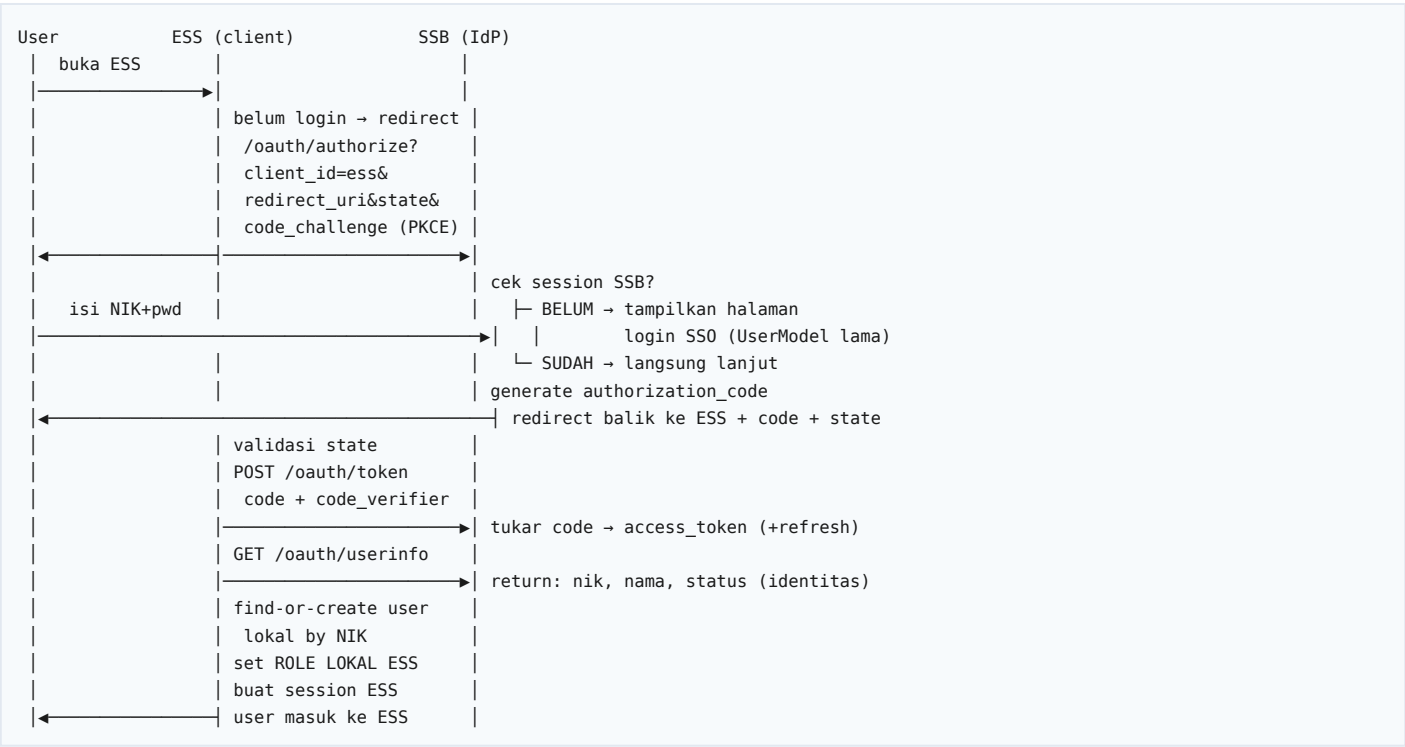
	SSB (IdP)	ESS / Warehouse (Client)
Autentikasi (siapa user)	pusat — validasi NIK + password	menerima hasil
Identitas user (NIK, nama)	sumber kebenaran	sinkron by NIK
Role / permission	tidak mengatur	dikelola lokal tiap app
Session aplikasi	session SSB (untuk SSO)	session lokal app sendiri

Karena role bersifat lokal, kontrak SSO menjadi sederhana: SSB tidak perlu tahu hak akses tiap app.

5. Arsitektur



6. Workflow Login (Authorization Code + PKCE)



Kunci true SSO: `/oauth/authorize` membaca **session SSB**. Sekali user login di salah satu app, app berikutnya tidak perlu ketik password lagi — langsung mendapat `authorization_code`.

7. Kontrak `/oauth/userinfo` (identitas saja)

```
{
  "sub": "12345",           // NIK = identitas universal antar app
  "nik": "12345",
  "name": "Budi Santoso",  // snapshot dari API karyawan
  "email": null,
  "is_active": true        // status kepegawaian (dari API karyawan)
}
```

Tidak ada field `roles`. Tiap app memetakan NIK → role lokalnya sendiri.

8. Workflow Logout

Tahap awal cukup **front-channel logout** (per app):

```
User klik logout di ESS
→ ESS hapus session lokal
→ redirect ke SSB /sso/logout?redirect=<ess>
→ SSB hapus session SSB + revoke token app tsb
→ redirect balik ke ESS
```

Single Logout (SLO) penuh — back-channel ke semua app — dapat menyusul di tahap lanjutan.

9. Penambahan database (semua tabel BARU)

- Tabel bawaan Passport (via `php artisan passport:install`): `oauth_clients`, `oauth_access_tokens`, `oauth_refresh_tokens`, `oauth_auth_codes`, `oauth_personal_access_clients`.
- Tabel milik sendiri:
 - `sso_clients` — metadata app client: nama, logo, **whitelist** `redirect_uri`, status aktif. Dipakai untuk admin panel kelola aplikasi yang boleh memakai SSO.

`personal_access_tokens` (Sanctum) **tetap dipakai** untuk API existing — tidak bentrok.

10. Yang dikelola di tiap Client (ESS / Warehouse)

1. Tabel `users` lokal minimal: `nik` (unik, kunci sinkron), `name`, `last_login`.
2. Tabel role/permission **milik app sendiri** (boleh Spatie lagi atau tabel sederhana), di-mapping ke `nik`.
3. Saat callback SSO: `find-or-create user by nik` → set role lokal app → buat session.
4. **First login** (NIK belum punya role di app): arahkan ke halaman "akses belum diberikan / hubungi admin app", atau auto-assign role default. (Keputusan masing-masing admin app.)

11. Checklist implementasi

Di SSB (IdP) — semua additive

- ☒ `composer require laravel/passport "^10.4"` (v10.4.2 — wajib seri 10.x untuk Laravel 8) **[Tahap 1]**
- ☒ `php artisan migrate` → tabel `oauth_*` terbuat **[Tahap 1]**
- ☒ `php artisan passport:keys` → kunci di `storage/oauth-*.key` (sudah ter-ignore via `/storage/*.key`) **[Tahap 1]**
- ☒ Tambah guard `oauth` di `config/auth.php` (driver `passport`; `web` / `api` TIDAK diubah) **[Tahap 1]**
- ☒ Route `/oauth/authorize` & `/oauth/token` (registrasi `Passport::routes()`); reuse session + `LoginController` existing untuk halaman login **[Tahap 2]**
- ☒ Publish view consent Passport + atur masa berlaku token (access 15 mnt, refresh 30 hari) **[Tahap 2]**
- ☒ Route `/api/oauth/userinfo` (guard `oauth`) → identitas saja (NIK, nama, email, `is_active`) dari `hrd_karyawan` **[Tahap 3]**
- ☒ Tabel `sso_clients` + admin panel CRUD (`/admin/sso/clients`, gate `manage-sso`) + flag **trusted** (skip consent first-party) **[Tahap 3]**
- ☒ Batasi route manajemen client bawaan Passport (`/oauth/clients`, `/oauth/personal-access-tokens` tidak diregistrasi) **[Tahap 3]**
- ☐ Route `/sso/logout` (front-channel logout) **[Tahap 5]**

Di tiap Client (ESS, Warehouse) — template tersedia di `docs/client-integration/laravel/`

- ☒ Client OAuth2 + PKCE (tanpa paket tambahan, pakai HTTP client bawaan) **[Tahap 4]**
- ☒ Config `client_id`, `client_secret`, `redirect_uri`, base URL SSB (`config-services.php`, `.env`) **[Tahap 4]**
- ☒ Callback: find-or-create by NIK + hook mapping role lokal + session (`SsbSsoController`) **[Tahap 4]**
- ☒ Route + tombol "Login dengan SSB" + catatan middleware `redirect-to-SSO` (`routes-web.php`) **[Tahap 4]**
- ☐ **Terapkan & uji di repo ESS** (tempel file, set `.env`, jalankan) — dikerjakan di sisi ESS

12. Keamanan (wajib)

- **PKCE** untuk semua client (lindungi authorization code).
- **Whitelist** `redirect_uri` per client — cegah open redirect.
- **HTTPS** wajib di semua endpoint.
- Parameter `state` anti-CSRF pada alur authorize.
- Access token **short-lived** + refresh token; **revoke** saat logout.
- **Rate-limit** `/oauth/token` dan halaman login SSO.
- Validasi status kepegawaian (`is_active`) — user nonaktif ditolak di IdP.

13. Roadmap bertahap (aman, bisa rollback)

Tahap	Output	Dampak ke app lama
1	Pasang Passport + tabel <code>oauth_*</code> + guard <code>oauth</code> (belum diaktifkan ke publik) — SELESAI	nol (terverifikasi)
2	Aktifkan <code>/oauth/authorize</code> & <code>/oauth/token</code> , reuse login session existing — SELESAI	nol (terverifikasi)
3	<code>/oauth/userinfo</code> (identitas) + <code>sso_clients</code> + admin panel + skip-consent trusted — SELESAI	nol (terverifikasi)
4 	Paket integrasi ESS disiapkan (<code>docs/client-integration/laravel/</code>) + IdP terverifikasi siap — penerapan di repo ESS	nol (terisolasi)
5	Warehouse menyusul + front-channel logout	nol

Tahap	Output	Dampak ke app lama
6	(Opsional) Single Logout penuh & migrasi API existing bertahap	dikontrol

Setiap tahap tidak menyentuh: login web lama, Sanctum API, ServiceToken, MediaController.

14. Catatan Implementasi Tahap 1 (selesai 2026-06-06)

Yang dikerjakan:

```
composer require laravel/passport "^10.4" # terpasang v10.4.2
php artisan migrate                       # buat 5 tabel oauth_*
php artisan passport:keys --force         # storage/oauth-private.key & oauth-public.key
```

Plus konfigurasi (additive) di `config/auth.php` :

```
'oauth' => [
    'driver' => 'passport',
    'provider' => 'users',
],
```

Keputusan teknis penting:

- Versi Passport dikunci ke seri 10.x** (`^10.4`). Passport 11+ butuh Laravel 9+, sedangkan project ini Laravel 8.83 → wajib v10.x.
- Passport::routes() SENGAJA belum dipanggil**. Akibatnya endpoint `/oauth/authorize` , `/oauth/token` , dll **belum terdaftar** (terverifikasi via `php artisan route:list`). Ini menjaga Tahap 1 benar-benar "belum aktif ke publik". Registrasi route = Tahap 2.
- User model TIDAK diubah**. Model masih memakai `Laravel\Sanctum\HasApiTokens` sehingga API existing (login Sanctum, ServiceToken, MediaController) tetap utuh. Alur Authorization Code menerbitkan token lewat OAuth server Passport, bukan lewat trait model, jadi trait Passport belum diperlukan.

Catatan lanjutan: bila nanti butuh helper token Passport pada model (mis. `$user->tokens()`), import dengan alias agar tidak bentrok dengan Sanctum: `use Laravel\Passport\HasApiTokens as HasPassportTokens;`

- Kunci enkripsi aman dari git** — `.gitignore` sudah memuat `/storage/*.key` sebelum perubahan ini, jadi `oauth-private.key` tidak akan ter-commit.

Verifikasi non-disruptif (sudah dijalankan):

- `route:list | grep oauth` → tidak ada route OAuth Passport (hanya `api/oauth2-callback` milik Swagger yang memang sudah ada).
- Route login lama (`/` , `api/login` , `hrd/auth/login`) tetap ada.
- Guard terdaftar: `web` , `api` , `oauth` , `sanctum` .

File yang berubah: `composer.json` , `composer.lock` , `config/auth.php` (+ `vendor/` dan `storage/oauth-*.key` yang ter-ignore git).

15. Catatan Implementasi Tahap 2 (selesai 2026-06-06)

Tujuan: mengaktifkan endpoint OAuth2 dan membuktikan alur authorize me-reuse halaman login yang sudah ada — tanpa mengubah `LoginController` .

Yang dikerjakan:

1. Registrasi route Passport + masa berlaku token di `app/Providers/AuthServiceProvider.php` :

```
use Laravel\Passport\Passport;

public function boot()
{
    $this->registerPolicies();
    // ... gate existing ...

    Passport::routes(); // aktifkan /oauth/*
    Passport::tokensExpireIn(now()->addMinutes(15)); // access token pendek
    Passport::refreshTokensExpireIn(now()->addDays(30)); // refresh token
    Passport::personalAccessTokensExpireIn(now()->addDays(7));
}
```

2. `php artisan vendor:publish --tag=passport-views` → layar consent di `resources/views/vendor/passport/`.

3. Membuat client uji (tipe **public/PKCE**, tanpa secret — sesuai prinsip keamanan):

```
php artisan passport:client --public --name="ESS (dev)" \
    --redirect_uri="http://localhost:8001/auth/ssb/callback" --no-interaction
# Client ID: 1 (redirect bisa diganti saat integrasi ESS nyata di Tahap 4)
```

Cara kerja reuse login (kenapa `LoginController` tak perlu diubah):

- Route `oauth/authorize` otomatis bermiddleware `web, auth` (terverifikasi via `gatherMiddleware()` → `web, auth`).
- Guest yang membuka `/oauth/authorize` dilempar oleh middleware `Authenticate` ke login utama (`APP_URL`), dan URL tujuan disimpan di session (`url.intended`).
- Setelah login, Laravel memakai `redirect()->intended()` yang **memprioritaskan** `url.intended` di atas `redirectTo()` (`/home`) milik `LoginController`, sehingga user kembali ke `/oauth/authorize` dan alur berlanjut.

Contoh URL authorize (yang nanti dipanggil ESS):

```
GET /oauth/authorize?
client_id=1&
redirect_uri=http://localhost:8001/auth/ssb/callback&
response_type=code&
scope=&
state=<random>&
code_challenge=<base64url(sha256(verifier))>&
code_challenge_method=S256
```

Verifikasi (sudah dijalankan):

- `route:list` → `oauth/authorize`, `oauth/token`, `oauth/token/refresh`, `oauth/tokens` muncul.
- Middleware `oauth/authorize` = `web, auth`.
- Client uji tersimpan: `id=1`, `name="ESS (dev)"`, `public(secret null)=yes`, `revoked=0`.
- App tetap boot; route & login existing tidak berubah.

Belum dikerjakan (sengaja, masuk Tahap 3):

- `/oauth/userinfo` (identitas dari API karyawan).
- Tabel `sso_clients` + admin panel + flag **trusted** untuk skip layar consent pada app first-party (saat ini consent screen Passport masih tampil).
- Pembatasan route manajemen client bawaan Passport (`/oauth/clients`, `/oauth/tokens`) — akan diganti admin panel sendiri.

File yang berubah: `app/Providers/AuthServiceProvider.php` (+ `resources/views/vendor/passport/**` hasil publish, + 1 row `oauth_clients` di DB).

16. Catatan Implementasi Tahap 3 (selesai 2026-06-06)

Tujuan: lengkapi IdP dengan endpoint identitas, manajemen client, dan skip-consent untuk app first-party.

A. Endpoint `/api/oauth/userinfo` (`routes/api.php` , `guard oauth`):

```
Route::middleware('auth:oauth')->get('/oauth/userinfo', 'OAuth\UserInfoController@show');
```

Controller `app/Http/Controllers/Oauth/UserInfoController.php` mengembalikan **identitas saja** (tanpa role — sesuai keputusan arsitektur):

```
{ "sub": "<nik>", "nik": "<nik>", "name": "<nm_lengkap>",  
  "email": "<nmemail>", "is_active": true }
```

`name / email / is_active` diambil dari relasi `karyawan (hrd_karyawan): nm_lengkap, nmemail, dan is_active = (tgl_resign IS NULL)` . URL final ada di bawah prefix `api` → `/api/oauth/userinfo` .

B. Tabel `sso_clients` + **model** (`database/migrations/..._create_sso_clients_table.php`):

- Kolom: `oauth_client_id` (FK→`oauth_clients`, cascade), `name`, `slug`, `description`, `logo`, `is_active`, `trusted` .
- Melengkapi `oauth_clients` (kredensial), bukan menggantikan.
- Model `App\Models\SsoClient` (relasi `oauthClient`) .

C. Skip-consent untuk client trusted:

- Model `App\Models\Passport\Client` meng-override `skipsAuthorization()` → `true` bila baris `sso_clients` punya `trusted = 1` .
- Didaftarkan via `Passport::useClientModel(...)` di `AuthServiceProvider` .

D. Admin panel (`/admin/sso/clients` , hanya super admin):

- Route resource di `routes/web.php` (prefix `admin/sso` , name `sso.*`) .
- Controller `OAuth\SsoClientController` (middleware `auth` + `can:manage-sso`) . Setiap aksi menyinkronkan `oauth_clients` (via `ClientRepository`) **dan** `sso_clients` . Secret confidential ditampilkan **sekali** saat dibuat.
- View di `resources/views/sso/clients/{index,create,edit}.blade.php` , `@extends('HRD.layouts.master')` (tema Iqonic) → tampil lengkap dengan topbar + sidebar HRD (markup `iq-card` , breadcrumb, alert `iq-alert`) .
- **Menu sidebar:** ditambahkan item "**Client SSO**" (ikon `ri-key-line`) di `resources/views/HRD/partials/template_one/_sidebar.blade.php` , dibungkus `@can('manage-sso')` → tampil untuk super admin (nik 999999999 atau role `super_admin / Super Admin`) , mengarah ke `route('sso.clients.index')` .
- Tidak perlu sentuh whitelist aset: `_body_header` HRD memuat seluruh aset tema tanpa guard route, jadi halaman `/admin/sso/*` otomatis dapat CSS/JS-nya. Terverifikasi render penuh (HTTP 200) dengan user super admin.
- Gate `manage-sso` (di `AuthServiceProvider`): super admin (nik 999999999 atau role `super_admin / Super Admin`) .

E. Batasi route Passport bawaan (security):

```
Passport::routes(function ($router) {  
    $router->forAuthorization();  
    $router->forAccessTokens();  
    $router->forTransientTokens();  
    // forClients() & forPersonalAccessTokens() SENGAJA tidak didaftarkan  
});
```

→ `/oauth/clients*` dan `/oauth/personal-access-tokens*` tidak ada, sehingga user biasa tidak bisa membuat client OAuth sendiri. Manajemen client hanya lewat admin panel.

Verifikasi (sudah dijalankan):

- `route:list` → `api/oauth/userinfo` + `sso.clients.*` muncul; `/oauth/clients` & `personal-access-tokens` **tidak ada**.
- `Passport::clientModel()` = `App\Models\Passport\Client`.
- Gate `manage-sso` terdaftar.
- Client uji (ID 1) didaftarkan ke `sso_clients` sebagai trusted → `skipAuthorization() === true`.

File baru/berubah: `database/migrations/..._create_sso_clients_table.php`, `app/Models/SsoClient.php`, `app/Models/Passport/Client.php`, `app/Http/Controllers/Oauth/UserInfoController.php`, `app/Http/Controllers/Oauth/SsoClientController.php`, `resources/views/sso/clients/*.blade.php`, `app/Providers/AuthServiceProvider.php`, `routes/web.php`, `routes/api.php`.

17. Catatan Implementasi Tahap 4 (2026-06-06)

Tujuan: menyiapkan integrasi pilot **ESS** (sisi client). Karena ESS adalah aplikasi terpisah, deliverable di repo IdP ini adalah **paket kode + panduan siap-tempel** + verifikasi bahwa IdP siap melayani alurnya.

Lokasi paket: `docs/client-integration/laravel/`

File	Fungsi
<code>README.md</code>	Panduan langkah-demi-langkah + diagram alur
<code>SsbSsoController.php</code>	Controller client: redirect (PKCE+state), callback (tukar token, userinfo, find-or-create by NIK, hook role lokal), logout lokal
<code>config-services.php</code>	Blok <code>services.ssb</code> (<code>base_url</code> , <code>client_id</code> , <code>secret</code> , <code>redirect</code>)
<code>routes-web.php</code>	Route <code>ssb.redirect</code> / <code>ssb.callback</code> / <code>ssb.logout</code>
<code>env.example.txt</code>	Variabel <code>.env</code> ESS
<code>migration_users_nik.php</code>	Skema user lokal ESS (kolom <code>nik</code> unik)

Pola teknis: OAuth2 Authorization Code + **PKCE (S256)**, validasi `state`, tanpa dependency tambahan (HTTP client bawaan Laravel). Role **tidak** diambil dari SSB — ada hook `mapLocalRole()` agar ESS menetapkan role lokalnya sendiri.

Client dev di SSB: "ESS (dev)" — `client_id=1`, public/PKCE, **trusted** (skip consent), redirect `http://localhost:8001/auth/ssb/callback`.

Verifikasi kesiapan IdP (via HTTP kernel, sudah dijalankan):

- `GET /oauth/authorize` (guest) → **302** ke halaman login existing (`APP_URL`).
- `GET /api/oauth/userinfo` tanpa token + `Accept: application/json` → **401** `{"message":"Unauthenticated."}`. (Client memanggil dgn `acceptJson()`.)

Belum dikerjakan (di luar repo ini): penerapan & uji end-to-end di repo ESS (tempel file, set `.env`, jalankan ESS:8001 ↔ SSB:8000).

18. Kontrol Akses (siapa boleh masuk) — terpusat di SSB

Kebijakan: Pembuatan & pengelolaan user dilakukan **di SSB**. **Punya role di SSB = berhak akses ESS dan seluruh client lain**. User tanpa role tidak bisa masuk SSB, sehingga otomatis tidak bisa SSO ke client mana pun. (Super admin `999999999` selalu boleh.)

Penegakan (dua lapis, semuanya di SSB):

1. **Saat login** — `LoginController::authenticated()` menolak user tanpa role (kecuali super admin). Karena `/oauth/authorize` me-reuse login ini, user tanpa role tidak akan pernah memperoleh `code`.

2. Di `/api/oauth/userinfo` (pengaman berlapis) — mengembalikan **HTTP 403** `access_denied` bila user tak punya role. Menutup kasus tepi seperti sesi lama atau role dicabut setelah token terbit.

Sisi client (ESS): tidak mengelola akses. Cukup:

- Tangani `403` dari `userinfo` → tampilkan "tidak memiliki akses".
- `firstOrCreate` user lokal by NIK (akses sudah dijamin SSB).
- `mapLocalRole()` **opsional** — hanya untuk role *internal* ESS (menu/fitur di dalam ESS), bukan gerbang akses.

Implikasi: untuk memberi/menarik akses seseorang ke ESS, cukup beri/cabut **role** user tersebut di SSB (Manajemen Pengguna). Berlaku serentak ke semua client.

19. Glosarium

- **IdP (Identity Provider):** aplikasi pusat yang mengautentikasi user — di sini SSB.
- **SP / Client (Service Provider):** aplikasi yang mempercayakan login ke IdP — ESS, Warehouse.
- **Authorization Code + PKCE:** alur OAuth2 paling aman untuk web app, code ditukar token via back-channel.
- **Front-channel logout:** logout via redirect browser (per app).
- **Single Logout (SLO):** logout serentak di semua app via back-channel.